LABORATORY OBSERVATION

Course: Full Stack Web Development
Course Code: 23CM4121
Branch: CSM
Section: B
Task No.: 2
Student Name: K. Saranya
Roll No.: A24126552090

---

1. TITLE

Build an Interactive Student Profile Manager Using JavaScript and DOM Manipulation

2. AIM

To build a webpage that collects student details through input fields and, on a button click, dynamically generates a styled student profile card using JavaScript DOM manipulation, without reloading the page.

3. OBJECTIVE

The objectives of this experiment are:

- To understand how to structure input fields and a button using HTML5 elements.
- To use a JavaScript class to model a real-world entity such as a Student.
- To select DOM elements using document.getElementById().
- To handle a button click using addEventListener("click", ...).
- To read and clean up values typed into input fields using .value and .trim().
- To validate that all fields are filled before creating a profile, using a conditional check and alert().
- To dynamically create and style elements using document.createElement().
- To append dynamically created elements to the page using appendChild().
- To reuse a helper function (createDetail) to avoid repetitive code when building each detail row.
- To apply CSS3 styling — gradients, box-shadow, hover/focus effects and a keyframe animation — for a polished, card-based interface.

4. THEORY

A JavaScript class is a blueprint for creating objects that share the same shape. The constructor runs once when a new object is created with the new keyword, and sets up its properties — here, a Student's name, rollNumber, department, and cgpa.

The DOM (Document Object Model) represents the page as a tree of elements that JavaScript can read and change. Instead of hardcoding the profile card in the HTML, this program builds it piece by piece at runtime, based on whatever the user typed into the input fields, and inserts it into an initially empty <section id="profileContainer">.

The key methods and concepts used in this program are:

- addEventListener("click", fn) runs a function every time the Create Student Profile button is clicked.
- document.getElementById() selects a specific element from the page by its id attribute.
- .value.trim() reads the current text in an input field and removes leading/trailing whitespace.
- A simple if condition checks whether any of the four trimmed values is an empty string; if so, an alert() is shown and the function returns early.
- document.createElement() and appendChild() create a new, empty element in memory and then attach it into the visible page.
- A reusable createDetail(label, value, extraClass) function builds one label/value row at a time, avoiding repeated DOM-creation code.
- container.innerHTML = "" clears any previously generated profile card before a new one is added.
- CSS :focus and :hover style an input or button only while it is actively selected or pointed at, and a slideIn keyframe animation fades and slides the new profile card into view.

The basic flow of the page is:

User fills the input fields → Create Student Profile button is clicked → input values are read and trimmed → empty fields are checked → a Student object is created → a profile card is built with DOM methods → the card is appended to the page.

5. ALGORITHM / PROCEDURE

1. Create three files: index.html, style.css and script.js, and link style.css in the head and script.js at the end of the body.

2. Add a form-style card with labelled inputs for Student Name, Roll Number, Department and CGPA, plus a Create Student Profile button.

3. Add an empty <section id="profileContainer"> where the generated profile card will be inserted.

4. In script.js, define a Student class with a constructor that stores name, rollNumber, department and cgpa.

5. Select the input fields, the button and the container element using document.getElementById().

6. Add a click event listener on the Create Student Profile button.

7. Inside the handler, read the current values from each input field using .value.trim().

8. Check whether any field is empty; if so, show an alert and stop (return) without creating a profile.

9. Create a new Student object using the values just read.

10. Clear any existing content in the container using profileContainer.innerHTML = "".

11. Create a profile card , and inside it a header (icon + heading) and a details section.

12. Write a createDetail() helper that builds one label/value row as a and returns it.

13. Call createDetail() once for each of Name, Roll No, Department and CGPA, and append each row to the details section.

14. Append the header and details section to the card, then append the card to the container so it appears on the page.

15. Save the files and open index.html in a browser to test with different input values.

6. PROGRAM

index.html

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <!-- Makes the webpage responsive on different screen sizes -->
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Student Profile Manager</title>
    <!-- Linking the external CSS file -->
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <!-- Main container for the Student Profile Manager -->
    <div class="container">
        <!-- Page heading -->
        <header>
            <h1>□ Student Profile Manager</h1>
            <p>Create and view a student profile dynamically using JavaScript</p>
        </header>
        <!-- Form section for entering student details -->
        <section class="form-card">
            <h2>Enter Student Details</h2>
            <div class="input-group">
                <label for="name">Student Name</label>
                <input type="text" id="name" placeholder="Enter student name">
            </div>
            <div class="input-group">
                <label for="roll">Roll Number</label>
                <input type="text" id="roll" placeholder="Enter roll number">
            </div>
            <div class="input-group">
                <label for="department">Department</label>
                <input type="text" id="department" placeholder="Enter department">
            </div>
            <div class="input-group">
                <label for="cgpa">CGPA</label>
                <input type="number" id="cgpa" step="0.01" min="0" max="10"
                       placeholder="Enter CGPA">
            </div>
            <!-- Button used to create the profile dynamically -->
            <button id="createBtn">Create Student Profile</button>
        </section>
        <!--
            This empty section acts as a container.
            JavaScript will dynamically create the student profile
            and insert it into this section.
        -->
        <section id="profileContainer"></section>
    </div>
    <!-- Linking the external JavaScript file -->
    <script src="script.js"></script>
</body>
</html>
```

style.css

```css
/* Remove default spacing and apply a common font */
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
}
/* Styling the complete webpage */
body {
    font-family: Arial, sans-serif;
```

```
    min-height: 100vh;
    background: linear-gradient(135deg, #667eea, #764ba2);
    display: flex;
    justify-content: center;
    align-items: center;
    padding: 30px;
}
/* Main application container */
.container {
    width: 100%;
    max-width: 850px;
}
/* Page heading */
header {
    text-align: center;
    color: white;
    margin-bottom: 25px;
}
header h1 {
    font-size: 32px;
    margin-bottom: 8px;
}
header p {
    font-size: 15px;
    opacity: 0.9;
}
/* Card containing the input form */
.form-card {
    background: white;
    padding: 30px;
    border-radius: 18px;
    box-shadow: 0 10px 30px rgba(0, 0, 0, 0.2);
    margin-bottom: 25px;
}
.form-card h2 {
    text-align: center;
    margin-bottom: 22px;
}
/* Each input field */
.input-group {
    margin-bottom: 16px;
}
.input-group label {
    display: block;
    font-weight: bold;
    margin-bottom: 6px;
}
.input-group input {
    width: 100%;
    padding: 12px;
    border: 1px solid #ccc;
    border-radius: 8px;
    font-size: 15px;
}
/* Input focus effect */
.input-group input:focus {
    outline: none;
    border-color: #667eea;
    box-shadow: 0 0 5px rgba(102, 126, 234, 0.4);
}
/* Create button */
button {
    width: 100%;
    padding: 13px;
    border: none;
    border-radius: 9px;
    background: #667eea;
    color: white;
    font-size: 16px;
```

```
    font-weight: bold;
    cursor: pointer;
    transition: 0.3s;
}
button:hover {
    background: #4f63c7;
    transform: translateY(-2px);
}
/* Dynamically generated student profile */
.profile-card {
    background: white;
    border-radius: 18px;
    overflow: hidden;
    box-shadow: 0 10px 30px rgba(0, 0, 0, 0.2);
    animation: slideIn 0.5s ease;
}
/* Profile heading */
.profile-header {
    background: linear-gradient(135deg, #667eea, #764ba2);
    color: white;
    text-align: center;
    padding: 25px;
}
.profile-header .icon {
    font-size: 45px;
    margin-bottom: 8px;
}
.profile-header h2 {
    font-size: 24px;
}
/* Student information section */
.profile-details {
    padding: 25px;
}
/* Individual student information row */
.detail {
    display: flex;
    justify-content: space-between;
    padding: 14px 5px;
    border-bottom: 1px solid #eee;
}
.detail:last-child {
    border-bottom: none;
}
.detail .label {
    font-weight: bold;
    color: #555;
}
.detail .value {
    color: #333;
    text-align: right;
}
/* CGPA highlight */
.cgpa {
    font-weight: bold;
    color: #667eea !important;
}
/* Animation for dynamically created profile */
@keyframes slideIn {
    from {
        opacity: 0;
        transform: translateY(20px);
    }
    to {
        opacity: 1;
        transform: translateY(0);
    }
}
/* Responsive design for smaller screens */
```

```css
@media (max-width: 600px) {
    body {
        padding: 15px;
    }
    header h1 {
        font-size: 25px;
    }
    .form-card {
        padding: 20px;
    }
    .detail {
        flex-direction: column;
        gap: 5px;
    }
    .detail .value {
        text-align: left;
    }
}
```

script.js

```js
// Creating a Student class
class Student {
    // Constructor initializes the properties of a student
    constructor(name, rollNumber, department, cgpa) {
        this.name = name;
        this.rollNumber = rollNumber;
        this.department = department;
        this.cgpa = cgpa;
    }
}
// Selecting the required HTML elements using their IDs
const nameInput = document.getElementById("name");
const rollInput = document.getElementById("roll");
const departmentInput = document.getElementById("department");
const cgpaInput = document.getElementById("cgpa");
const createButton = document.getElementById("createBtn");
const profileContainer = document.getElementById("profileContainer");
// Adding a click event to the Create Profile button
createButton.addEventListener("click", function () {

    const name = nameInput.value.trim();
    const rollNumber = rollInput.value.trim();
    const department = departmentInput.value.trim();
    const cgpa = cgpaInput.value.trim();
    // Check whether all fields are filled
    if (name === "" || rollNumber === "" ||
        department === "" || cgpa === "") {
        alert("Please enter all student details.");
        return;
    }

    const student = new Student(
        name,
        rollNumber,
        department,
        cgpa
    );

    profileContainer.innerHTML = "";

    const profileCard = document.createElement("div");
    profileCard.className = "profile-card";
    // Creating the profile heading section
    const profileHeader = document.createElement("div");
    profileHeader.className = "profile-header";
    const icon = document.createElement("div");
    icon.className = "icon";
    icon.textContent = "";
```

```
    const heading = document.createElement("h2");
    heading.textContent = "Student Profile";
    // Adding heading elements to the profile header
    profileHeader.appendChild(icon);
    profileHeader.appendChild(heading);
    // Creating the student details section
    const profileDetails = document.createElement("div");
    profileDetails.className = "profile-details";

    function createDetail(labelText, valueText, extraClass = "") {
        const detail = document.createElement("div");
        detail.className = "detail";
        const label = document.createElement("span");
        label.className = "label";
        label.textContent = labelText;
        const value = document.createElement("span");
        value.className = "value " + extraClass;
        value.textContent = valueText;
        detail.appendChild(label);
        detail.appendChild(value);
        return detail;
    }

    profileDetails.appendChild(
        createDetail("Name", student.name)
    );
    profileDetails.appendChild(
        createDetail("Roll No", student.rollNumber)
    );
    profileDetails.appendChild(
        createDetail("Department", student.department)
    );
    profileDetails.appendChild(
        createDetail("CGPA", student.cgpa, "cgpa")
    );

    profileCard.appendChild(profileHeader);
    profileCard.appendChild(profileDetails);
    profileContainer.appendChild(profileCard);
});
```

7. CODE EXPLANATION

Code	Explanation
`class Student { constructor(...) }`	Defines a blueprint for creating student objects with name, rollNumber, department, and cgpa.
`new Student(...)`	Creates a new object instance using the values read from the input fields.
`document.getElementById()`	Selects a specific element from the page by its id attribute.
`addEventListener("click", fn)`	Runs the given function every time the Create Student Profile button is clicked.
`.value.trim()`	Reads the current text typed into an input field and strips extra whitespace.
`if (... === "") { alert(...); return; }`	Validates that all four fields are filled before a profile is created; stops the function early otherwise.
`profileContainer.innerHTML = ""`	Clears any previously generated profile card before adding a new one.
`document.createElement()`	Creates a new, empty HTML element in memory.
`element.textContent`	Sets the visible text inside an element.
`appendChild()`	Attaches a created element as a child of another element in the DOM.
`createDetail(label, value, extraClass)`	Reusable helper that builds one label/value detail row, avoiding repeated code for each field.
`:focus / :hover`	Styles an input while it is being typed into, or the button while the pointer is over it.

Code	Explanation
`@keyframes slideIn`	Animates the profile card with a fade-and-slide-up effect when it first appears.

8. TEST CASES AND EXECUTION DATA

The page was opened in a browser and tested with different inputs; the results were recorded below.

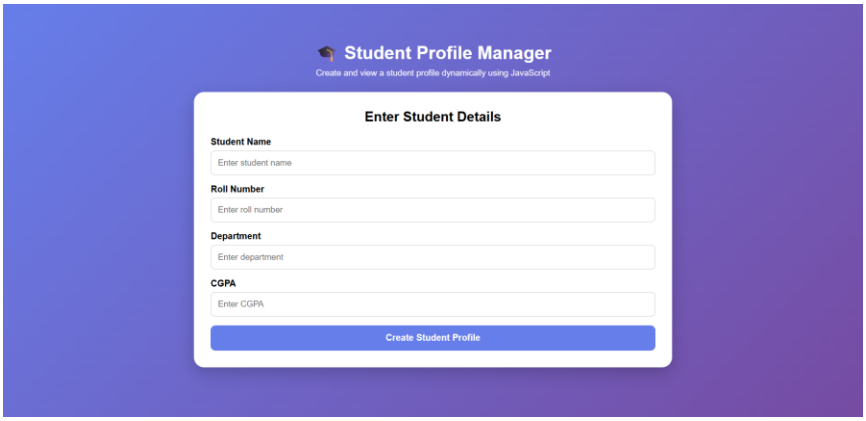
Test Case	Action	Expected Result	Actual Result	Status
TC-01	Open index.html in a browser	Empty form displayed with 4 labelled fields and a Create Student Profile button	Form displayed correctly	Pass
TC-02	Click Create Student Profile with all fields empty	An alert asks to enter all student details; no profile card is created	Alert shown as expected	Pass
TC-03	Fill all fields and click Create Student Profile	A profile card appears below the form with the entered values; page does not reload	Profile card displayed correctly	Pass
TC-04	Enter a CGPA above 10	The number input's max=10 restricts the value entered	Value restricted as expected	Pass
TC-05	Click Create Student Profile a second time with different values	Old profile card is removed and replaced with the new one	Card replaced correctly	Pass
TC-06	Click into any input field	Border colour changes and a soft glow appears on focus	Focus effect displayed	Pass

Additional Validation Test

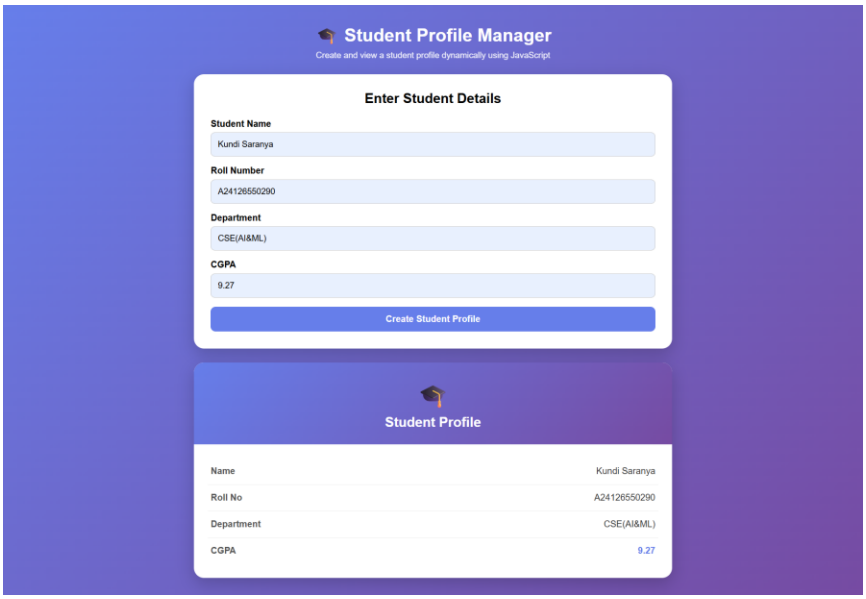
Test Case	Input / Action	Expected Result	Status
TC-07	Hover over the Create Student Profile button	Background darkens and the button lifts slightly	Pass
TC-08	Fill only some fields (e.g. leave CGPA empty) and click Create Student Profile	Alert asks to enter all student details; no profile card is created	Pass

9. OUTPUT

Output 1: Empty Student Details Form



Output 2: Filled Form and Generated Profile Card (after clicking Create Student Profile)



10. RESULT

Thus, an interactive Student Profile Manager was successfully built using HTML, CSS and JavaScript DOM manipulation. Clicking the Create Student Profile button with all fields filled correctly created a Student object and rendered a styled profile card, complete with a gradient header, icon, and highlighted CGPA value, without reloading the page. Leaving any field empty correctly triggered a validation alert instead of creating a profile.